

Calibration d'une famille paramétrique de volatilité locale

Application au SX5E - Euro Stoxx 50

Thi Hoang Ha DO

M2 Finance - Marchés Financiers & Banque d'Investissement

Université Paris Dauphine - PSL

Cours : Calcul Stochastique - Emmanuel Lépinette

Juin 2026

Table des matières

1	Problématique	3
1.1	Contexte	3
1.2	Support IA	3
2	Données de marché	3
2.1	Données Bloomberg	3
3	Modèle stochastique sous probabilité risque-neutre Q	4
3.1	Structure du marché	4
3.2	Estimation de taux sans risque sur l'€STR	4
4	Famille polynomiale	5
4.1	Définition de la famille	5
4.2	Calibration	6
4.3	Analyse de la surface calibrée	6
5	Famille exponentielle	9
5.1	Définition de la famille	9
5.2	Calibration	9
6	Conclusion	12
6.1	Synthèse des performances	12
6.2	Avantages et limites de chaque modèle	13
6.3	Utilisation des modèles	14
6.4	Pistes pour réduire les biais	15
	Annexes	16
A	Code complet	16

1 Problématique

1.1 Contexte

Dans quelle mesure une famille paramétrique de volatilité locale, polynomiale ou exponentielle, permet-elle de calibrer fidèlement les prix d'options vanilles observés sur le marché, tout en restant interprétable et robuste hors échantillon ?

Dans ce projet, deux familles sont comparées : une famille polynomiale bivariée et une famille exponentielle :

$$\sigma_{LV} = \sum_{i=0}^n \sum_{j=0}^m \alpha_{i,j} S_t^i t^j \quad (1)$$

$$\sigma_{LV} = \beta_0 + \beta_1 \exp - \frac{(S_t - \mu)^2}{2\sigma^2 t} \quad (2)$$

1.2 Support IA

Pour l'utilisation de l'IA, j'ai utilisé Claude pour structurer et déboguer les codes Python. Elle m'a également aidé à générer les graphiques et implémenter les calibrations via le package scipy.

2 Données de marché

2.1 Données Bloomberg

Les données sont issues de Bloomberg au 09/06/2026 : 4906 options calls et puts sur l'Euro Stoxx 50 (SX5E), pour des maturités couvrant de 1 semaine à 22 mois. Le spot SX5E est de **6071,4** points. La query Bloomberg utilisée est de la forme :

```
=@BQL("options('SX5E Index', put_call in ['Call','Put'], expire_dt>=1w, expire_dt<=2y); "
      id_security_des(),value, expire_dt(),value, strike_px(),value, pct_moneyness(),value,
      px_last(),value"; "showids=f";"cols=5;rows=4906")
```

Listing 1 – Query Bloomberg

Le taux sans risque utilisé est l'*STR*, avec $r_0 = 1,931\%$ au 09/06/2026.

Sur certaines maturités et certains strikes, en particulier les options très proches de l'expiration ou fortement hors de la monnaie, Bloomberg retourne $\#N/A$ à la place du prix, ces instruments ne sont pas cotés à un spread bid-ask exploitable ce jour-là. Ces 82 lignes sont éliminées lors du traitement de données (exception levée par la conversion `float('#N/A')`), ce qui ramène le nombre d'options à 4824 valides. Des filtres de liquidité complémentaires sont ensuite appliqués : $\text{prix} > 0$, maturité $T \geq 0,01$ an, moneyness $K/S_0 \in [0, 200]$.

La volatilité implicite est calculée par inversion numérique de la formule de Black-Scholes via `scipy.optimize.minimize_scalar`, qui minimise

$$(BS(\sigma) - \text{prix})^2 \quad \text{sur } \sigma \in [10^{-4}, 5] \quad (3)$$

où $BS(\sigma)$ est le Black-Scholes pricing modèle pour le Call Option :

$$C(S, K, T, r, \sigma) = S \mathcal{N}(d_1) - K e^{-rT} \mathcal{N}(d_2), \quad (4)$$

avec

$$d_1 = \frac{\log(S/K) + (r + \sigma^2/2) T}{\sigma \sqrt{T}}, \quad d_2 = d_1 - \sigma \sqrt{T} \quad (5)$$

Un filtre final retient uniquement les volatilités implicites dans l'intervalle (0%, 150%) afin d'éliminer les solutions numériques aberrantes. Sur la surface observée, on note un skew négatif caractéristique : la volatilité implicite est significativement plus élevée pour les puts OTM ($K < \text{Spot}$) ce qui reflète la demande de couverture contre les baisses. La structure par terme est croissante sur les maturités courtes. La volatilité implicite ATM moyenne (moneyness $\in [0,97; 1,03]$) est d'environ 18%.

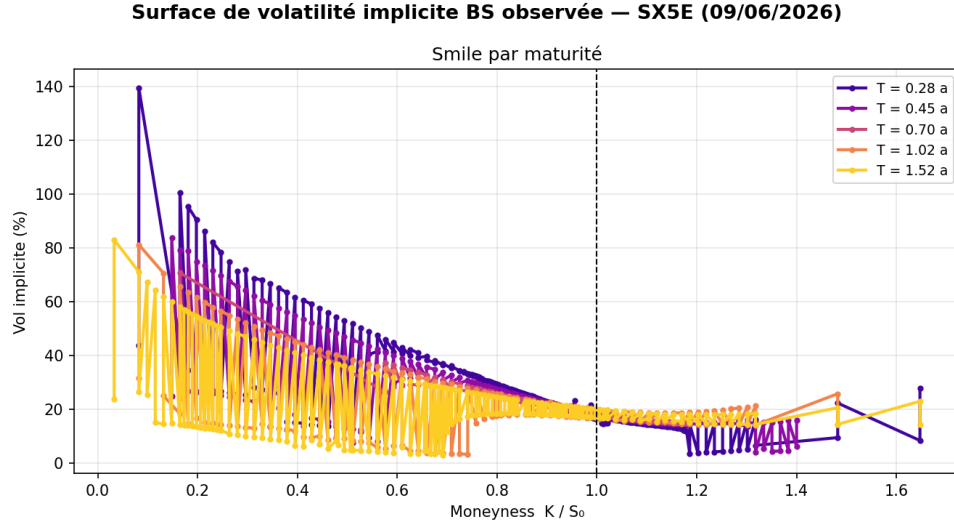


FIGURE 1 – Surface de Volatilité implicite Black-Scholes observée - SX5E.

3 Modèle stochastique sous probabilité risque-neutre Q

3.1 Structure du marché

Le marché est composé d'un actif sans risque S_0 et d'un actif risqué S . Sous la probabilité risque-neutre Q (obtenue par le théorème de Girsanov), la dynamique jointe est :

$$dr_t = \alpha(\beta - r_t) dt + \gamma\sqrt{|r_t|} dW_t^r \quad [\text{taux sans risque}] \quad (6)$$

$$dS_t^0 = r_t S_t^0 dt, \quad S_0^0 = 1 \quad [\text{actif sans risque}] \quad (7)$$

$$d\tilde{S}_t = \sigma_{LV}(t, \tilde{S}_t) \tilde{S}_t dB_t, \quad B \perp W \quad [\text{valeur actualisée}] \quad (8)$$

$$S_t = \tilde{S}_t \cdot S_t^0 \cdot S_0 \quad [\text{prix réel}] \quad (9)$$

Le drift de la valeur actualisée $\tilde{S}$ est nul sous Q : c'est une martingale. Le prix d'un dérivé de payoff $g(S_T)$ à maturité T est :

$$V_0 = \mathbb{E}^Q \left[g(S_T) \cdot e^{-\int_0^T r_u du} \right] \quad (10)$$

3.2 Estimation de taux sans risque sur l'€STR

Les paramètres du modèle d'estimation de taux sans risque sont estimés par maximum de vraisemblance (MLE) sur les données €STR quotidiennes de janvier 2023 à juin 2026. La discrétisation d'Euler du modèle donne :

$$\begin{aligned}
t_i^n &= \frac{T}{n \cdot i} \quad \text{avec } i \in [0, n] \\
r_{t_i^n} &= r_{t_{i-1}^n} + a(b - r_{t_{i-1}^n}) \frac{T}{n} + \gamma \Delta W_{i-1}^n \quad \text{avec } i \in [0, n]
\end{aligned} \tag{11}$$

On modélise l'incrément $dr_t \sim N(\text{drift}, \text{variance})$ avec :

$$\text{drift} = \alpha(\beta - r_t)\Delta t, \quad \text{variance} = \gamma^2 |r_t| \Delta t \tag{12}$$

Les trois paramètres s'interprètent ainsi :

- α (alpha) : vitesse de retour à la moyenne, plus α est grand, plus le taux revient rapidement vers son niveau d'équilibre β .
- β (beta) : niveau de taux à long terme vers lequel le taux converge.
- γ (gamma) : volatilité du taux d'intérêt, contrôle l'amplitude des fluctuations de r_t .

ON cherche les estimateurs qui minimise la log-vraisemblance négative du CIR :

$$\log p(r_{t+\Delta t} \mid r_t, a, b, \gamma) = -\frac{1}{2} \log(2\pi \gamma^2 r_t \Delta t) - \frac{(r_{t+\Delta t} - r_t - a(b - r_t)\Delta t)^2}{2 \gamma^2 r_t \Delta t} \tag{13}$$

$$\hat{a}, \hat{b}, \hat{\gamma} = \arg \min_{a, b, \gamma} \left[-\sum_{t=1}^N \log p(r_{t+\Delta t} \mid r_t, a, b, \gamma) \right] \tag{14}$$

La log-vraisemblance négative est minimisée via `scipy.optimize.differential_evolution` sur les bornes $\alpha \in [0, 01; 10]$, $\beta \in [0, 1\%; 10\%]$, $\gamma \in [0, 1\%; 50\%]$

Les valeurs obtenues sont :

$$\begin{aligned}
\hat{\alpha} &= 0,3186 \\
\hat{\beta} &= 0,029 \\
\hat{\gamma} &= 0,0374
\end{aligned} \tag{15}$$

Choix de l'€STR vs Euribor : sous Q , l'actualisation se fait au taux sans risque de défaut. L'€STR (taux BCE overnight, publié quotidiennement depuis 2019 et taux de référence officiel depuis la réforme des indices en 2022) est exempt de prime de crédit interbancaire, contrairement à l'Euribor qui incorpore un spread de liquidité et de crédit entre banques. Son utilisation est donc cohérente avec le cadre théorique.

4 Famille polynomiale

4.1 Définition de la famille

La famille polynomiale est définie par un développement en base monomiale bivariée sur le moneyness $x = K/S_0$ et la maturité t .

$$\sigma_{LV} = \sum_{i=0}^n \sum_{j=0}^m \alpha_{i,j} S_t^i t^j \tag{16}$$

avec n, m sont des coefficients à calibrer.

4.2 Calibration

La calibration se fait à la date $t = 0$ sur des données de marché observées. La variable d'état pertinente est le strike relatif au spot initial : $x = K/Spot$, c'est-à-dire le moneyness. Utiliser S_t supposerait de connaître la trajectoire future du sous-jacent, impossible au moment de la calibration. De plus, le moneyness est adimensionnel et centré autour de 1 (ATM), ce qui améliore le conditionnement numérique de la régression, les puissances x^i restent d'ordre 1 dans la zone d'intérêt, contrairement à des puissances de S_t qui seraient de l'ordre de 6000^i .

La calibration minimise l'erreur quadratique sur les volatilités implicites observées :

$$\min_{\alpha} \sum_{i=1}^N (\sigma_{LV}(t_i, x_i; \alpha) - IV_i)^2 + \lambda \|\alpha\|^2 \quad (17)$$

$$\begin{aligned} \text{AIC} &= N \cdot \log(\text{RSS}/N) + 2k \\ \text{BIC} &= N \cdot \log(\text{RSS}/N) + \log(N) \cdot k \end{aligned} \quad (18)$$

TABLE 1 – Sélection du degré polynomial (n, m)

n	m	#params	RMSE%	MAE%	% $\sigma < 0$	AIC	BIC
1	1	4	2.691	1.945	0.00%	-21546.8	-21522.8
1	2	6	2.512	1.838	0.00%	-21952.5	-21916.5
1	3	8	2.469	1.808	0.00%	-22052.5	-22004.5
2	1	6	2.079	1.301	0.00%	-23080.3	-23044.3
2	2	9	1.758	1.143	0.00%	-24074.6	-24020.6
2	3	12	1.654	1.087	0.00%	-24433.2	-24361.2
3	1	8	1.929	1.205	0.00%	-23522.3	-23474.3
3	2	12	1.639	1.038	0.00%	-24487.8	-24415.8
3	3	16	1.501	0.934	0.00%	-25001.3	-24905.3
4	1	10	1.924	1.201	0.00%	-23533.5	-23473.5
4	2	15	1.624	1.028	0.00%	-24536.3	-24446.3
4	3	20	1.488	0.922	0.00%	-25046.8	-24926.8

Le degré $n=4, m=3$ minimise à la fois l'AIC (-25046.8) et le BIC (-24926.8).

4.3 Analyse de la surface calibrée

Skew négatif : La surface reproduit fidèlement l'asymétrie caractéristique du SX5E : IV moyenne de :

- 33.8% pour les puts OTM ($K/Spot < 0,90$)
- 18% ATM
- 14,9% pour les calls OTM ($K/Spot > 1,05$).

Ce skew négatif reflète la demande structurelle de protection contre les baisses (achat de puts OTM par les investisseurs institutionnels), qui fait monter leur prix et donc leur IV implicite.

Structure par terme différenciée selon le moneyness : La structure par terme n'est pas uniforme, elle dépend fortement du strike :

- Puts OTM ($K/Spot \approx 0,85$) : structure décroissante : 30.3% à $T \approx 0,1$ an ; 25,8% à $T \approx 1,5$ an. La prime de protection est plus chère à court terme, où le risque de gap baissier est plus concentré temporellement.

- ATM ($K/Spot \approx 1$) : structure légèrement croissante : 17,3% à $T \approx 0,1$ an ; 18,4% à $T \approx 1,5$ an. La prime de risque à long terme s'ajoute progressivement.
- Calls OTM ($K/Spot > 1,10$) : structure modérément croissante, avec une prime de terme liée à l'incertitude sur la direction du marché..

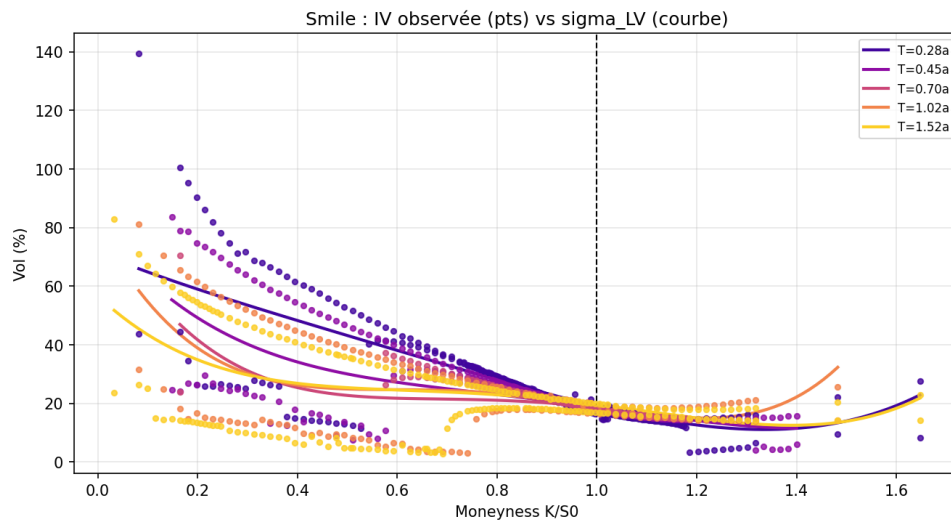


FIGURE 2 – Smile IV observée - famille Polynomiale.

Analyse des résidus de pricing : La validation porte sur 25 options en in-sample et 20 en out-of-sample, couvrant 5 niveaux de moneyness ($K/Spot \in \{0,75; 0,90; 1,00; 1,10; 1,20\}$) et plusieurs maturités.

TABLE 2 – Bias de modèle Polynomiale (n, m)

Modèle	Échantillon	Biais moyen
Polynomiale	In-sample	+19,2%
Polynomiale	Out-of-sample	+19,8%

Les résidus présentent un biais positif systématique ($P_{MC} > P_{obs}$), qui s'explique par deux effets distincts selon la zone de moneyness :

- **Puts OTM ($K/Spot \leq 0,90$)** : erreur est faible, le modèle valide sur cette zone. Le biais est quasi-nul (+0,24% en global ; +0,55% in-sample ; -0,13 out-sample) la polynomiale capture fidèlement la dynamique là où elle a été calibrée.

TABLE 3 – Comparaison prix observés / prix Monte Carlo - Puts OTM ($K/\text{Spot} \leq 0,90$)

T	K	P_obs	P_MC	err.pct	modèle	échantillon
0,27	4550	1561	1552,23	-0,57	poly	in-sample
0,44	4550	1587	1576,15	-0,65	poly	in-sample
0,69	4550	1627	1610,50	-1,01	poly	in-sample
0,69	5450	842,2	838,81	-0,40	poly	in-sample
1,02	4600	1568	1610,23	2,69	poly	in-sample
1,52	4550	1689	1704,94	0,95	poly	in-sample
1,52	5450	985,9	1014,32	2,88	poly	in-sample
0,35	4550	1568	1562,92	-0,33	poly	out-sample
0,60	4550	1612	1597,23	-0,89	poly	out-sample
0,60	5450	814,9	813,08	-0,22	poly	out-sample
0,85	4600	1599	1586,57	-0,76	poly	out-sample
0,85	5450	877,4	879,22	0,20	poly	out-sample
1,27	4600	1614	1634,10	1,22	poly	out-sample

- **ATM ($K/\text{Spot} \approx 1$)** : biais modéré (+10,21% en global ; +10,67% in-sample ; 9,62 out-sample). Cette surestimation est structurelle : il reflète la différence entre la volatilité locale polynomiale calibrée sous Black-Scholes flat et la dynamique effective sous le modèle CIR + vol locale utilisé dans le pricer MC.

TABLE 4 – Comparaison prix observés / prix Monte Carlo - ATM ($K/\text{Spot} \approx 1$)

T	K	P_obs	P_MC	err.pct	modèle	échantillon
0,28	6075	237,00	267,55	12,89	poly	in-sample
0,45	6075	312,60	343,28	9,82	poly	in-sample
0,70	6050	414,60	445,70	7,50	poly	in-sample
1,02	6100	444,30	513,96	15,68	poly	in-sample
1,52	6050	600,70	645,78	7,50	poly	in-sample
0,35	6075	269,70	303,38	12,49	poly	out-sample
0,60	6050	381,90	413,50	8,28	poly	out-sample
0,85	6050	458,40	493,94	7,75	poly	out-sample
1,27	6100	514,80	566,16	9,98	poly	out-sample

- **Calls OTM profonds ($K/\text{Spot} \geq 1,10$)** : les biais sont très élevés (+40,64% en global ; +38,70% in-sample ; 42,84 out-sample), le modèle n'est pas fiable sur cette zone. L'écart est amplifié par le fait que ces strikes sont peu représentés dans le dataset de calibration, la polynomiale y extrapole avec une σ_{LV} surestimée. Ces options très OTM, peu liquides, ne constituent pas un test robuste du modèle.

TABLE 5 – Comparaison prix observés / prix Monte Carlo - Calls OTM profonds, in-sample

T	K	P_obs	P_MC	err.pct	modèle	échantillon
0,28	7300	1,60	1,96	22,48	poly	in-sample
0,45	7300	8,40	14,00	66,73	poly	in-sample
0,70	6700	127,10	170,59	34,22	poly	in-sample
0,70	7150	43,40	68,52	57,87	poly	in-sample
1,02	6700	186,30	253,37	36,00	poly	in-sample
1,02	7300	60,60	98,32	62,24	poly	in-sample
1,52	6700	315,00	344,34	9,31	poly	in-sample
1,52	7300	131,90	159,35	20,81	poly	in-sample
0,35	7300	3,50	5,67	62,03	poly	out-sample
0,60	6700	102,80	142,80	38,91	poly	out-sample
0,60	7150	33,00	50,40	52,72	poly	out-sample
0,85	6700	162,50	212,86	30,99	poly	out-sample
0,85	7300	45,50	73,20	60,89	poly	out-sample
1,27	6700	243,90	297,43	21,95	poly	out-sample
1,27	7300	95,10	125,90	32,39	poly	out-sample

5 Famille exponentielle

5.1 Définition de la famille

La famille exponentielle retenue est une gaussienne en log-moneyness, définie par :

$$\sigma_{LV} = \beta_0 + \beta_1 \exp - \frac{(S_t - \mu)^2}{2\sigma^2 t} \quad (19)$$

Avec :

- σ : volatilité réalisée annualisée du SX5E, estimée sur données historiques et fixée avant toute calibration.
- $\mu = 1$: centre de la gaussienne en moneyness normalisé, fixé à ATM (correspondant à $K = S_0$).
- β_0 : vol plancher, calibrée sur les options, valeur minimale que prend σ_{LV} loin de ATM.
- β_1 : amplitude du smile, calibrée sur les options.

Estimation des paramètres fixes sur données historiques : Les log-rendements journaliers du SX5E sur la période juin 2021 - juin 2026 (1 255 jours de trading) donnent :

$$\sigma_{hist} = 17,53\%/an \quad (20)$$

σ_{hist} est ensuite utilisé comme paramètre fixe dans σ_{LV} , il détermine la largeur de la gaussienne et n'est plus ajusté lors de la calibration sur les options.

5.2 Calibration

Seuls β_0 et β_1 sont calibrés par moindres carrés sur les volatilités implicites observées :

$$\min_{\beta_0, \beta_1} \sum_{i=1}^N (\sigma_{LV}(t_i, x_i) - IV_i)^2 \quad \text{sous } \beta_0 > 0, \beta_1 > 0 \quad (21)$$

L'optimisation utilise *scipy.optimize.differential_evolution*. Le résultat obtenu est :

$$\begin{aligned} \beta_0 &= 21,03\% \\ \beta_1 &= 0,10\% \end{aligned} \quad (22)$$

La valeur quasi-nulle de β_1 est révélatrice : l'amplitude calibrée est pratiquement nulle, ce qui signifie que le modèle se réduit à une volatilité constante $\sigma_{LV} \approx \beta_0 = 21\%$ sur toute la surface. Cela s'explique par une **limite structurelle de la forme fonctionnelle** : la gaussienne centrée en ATM ($\mu = 1$) est symétrique en moneyness, or le skew du SX5E est fortement asymétrique, la volatilité monte bien plus vite côté puts OTM ($x < 1$) que côté calls OTM ($x > 1$).

Toutes les valeurs calibrées sont quasi-identiques ($\sim 21\%$) indépendamment du moneyness et de la maturité, la surface est plate.

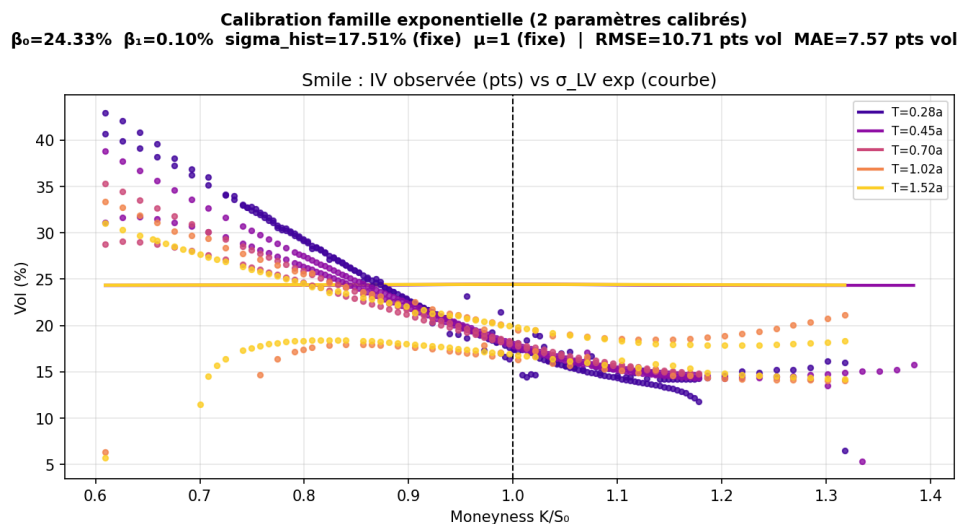


FIGURE 3 – Smile IV observée - famille exponentielle.

Résultats pricing :

Modèle	Échantillon	Biais moyen
Exponentielle	In-sample	+101,96%
Exponentielle	Out-of-sample	+90,87%

Les erreurs de la famille exponentielle sont sans commune mesure avec celles de la **polynomiale**, avec un biais de +101% in-sample et une erreur maximale de +942%. L'analyse par moneyness révèle la cause :

Zone	Exp biais
$K/S_0 \approx 0,75$ (deep ITM)	-0,39%
$K/S_0 \approx 0,90$ (ITM)	+3,12%
$K/S_0 \approx 1,00$ (ATM)	+22,82%

Zone	Exp biais
K/S ₀ ≈ 1,10 (OTM)	+73,95%
K/S ₀ ≈ 1,20 (deep OTM)	+406,09%

Sur les zones ITM et deep ITM ($x \leq 0,90$), les deux modèles exponentielle et polynomiale sont comparables (biais < 5%), la valeur intrinsèque domine et la volatilité locale a peu d'impact. La divergence apparaît dès ATM et s'amplifie sur les calls OTM : la famille exponentielle, réduite à une surface plate à 21%, surestime massivement la volatilité des calls OTM ($\sigma_{LV} = 21\%$ alors que l'IV observée est autour de 14.5%), ce qui gonfle les prix MC.

La dégradation est aussi visible par maturité : le biais exponentiel est de +230% à $T \approx 0,28$ an et décroît jusqu'à +42% à $T \approx 1,52$ an, la gaussienne avec σ_{hist} fixé est trop étroite à courte maturité (terme $\sigma_{hist}^2 \cdot t$ petit $\rightarrow$ exponentielle quasi-nulle), rendant la surface encore plus plate à court terme.

La famille exponentielle avec la volatilité historique fixe ne parvient pas à reproduire le skew asymétrique du SX5E.

T	K	P_obs	P_MC	err.pct	modèle	échantillon
0,28	4550	1561,20	1548,30	-0,83	exponentielle	in-sample
0,28	5475	692,80	682,76	-1,45	exponentielle	in-sample
0,28	6075	237,00	286,25	20,78	exponentielle	in-sample
0,28	6675	29,40	84,49	187,37	exponentielle	in-sample
0,28	7300	1,60	16,67	941,65	exponentielle	in-sample
0,45	4550	1586,50	1568,50	-1,13	exponentielle	in-sample
0,45	5475	750,50	746,02	-0,60	exponentielle	in-sample
0,45	6075	312,60	370,96	18,67	exponentielle	in-sample
0,45	6675	71,60	151,23	111,21	exponentielle	in-sample
0,45	7300	8,40	48,34	475,50	exponentielle	in-sample
0,70	4550	1626,90	1603,77	-1,42	exponentielle	in-sample
0,70	5450	842,20	846,92	0,56	exponentielle	in-sample
0,70	6050	414,60	483,60	16,64	exponentielle	in-sample
0,70	6700	127,10	231,53	82,16	exponentielle	in-sample
0,70	7150	43,40	128,07	195,10	exponentielle	in-sample
1,02	4600	1568,00	1611,82	2,79	exponentielle	in-sample
1,02	5500	833,70	910,44	9,21	exponentielle	in-sample
1,02	6100	444,30	570,13	28,32	exponentielle	in-sample
1,02	6700	186,30	333,38	78,95	exponentielle	in-sample
1,02	7300	60,60	180,80	198,36	exponentielle	in-sample
1,52	4550	1688,90	1741,56	3,12	exponentielle	in-sample
1,52	5450	985,90	1075,45	9,08	exponentielle	in-sample
1,52	6050	600,70	739,43	23,10	exponentielle	in-sample
1,52	6700	315,00	472,33	49,94	exponentielle	in-sample

T	K	P_obs	P_MC	err.pct	modèle	échantillon
1,52	7300	131,90	297,71	125,71	exponentielle	in-sample
0,35	4550	1568,20	1556,83	-0,73	exponentielle	out-sample
0,35	5475	715,00	711,48	-0,49	exponentielle	out-sample
0,35	6075	269,70	326,13	20,92	exponentielle	out-sample
0,35	6675	46,90	114,67	144,49	exponentielle	out-sample
0,35	7300	3,50	29,46	741,59	exponentielle	out-sample
0,60	4550	1611,60	1589,62	-1,36	exponentielle	out-sample
0,60	5450	814,90	816,49	0,19	exponentielle	out-sample
0,60	6050	381,90	447,17	17,09	exponentielle	out-sample
0,60	6700	102,80	199,26	93,83	exponentielle	out-sample
0,60	7150	33,00	103,37	213,25	exponentielle	out-sample
0,85	4600	1598,80	1582,21	-1,04	exponentielle	out-sample
0,85	5450	877,40	893,70	1,86	exponentielle	out-sample
0,85	6050	458,40	537,83	17,33	exponentielle	out-sample
0,85	6700	162,50	280,75	72,77	exponentielle	out-sample
0,85	7300	45,50	140,13	207,97	exponentielle	out-sample
1,27	4600	1614,30	1655,65	2,56	exponentielle	out-sample
1,27	5500	898,90	979,40	8,95	exponentielle	out-sample
1,27	6100	514,80	645,56	25,40	exponentielle	out-sample
1,27	6700	243,90	404,99	66,05	exponentielle	out-sample
1,27	7300	95,10	239,51	151,85	exponentielle	out-sample

TABLE 6 – Comparaison prix observés / prix Monte Carlo - famille exponentielle

6 Conclusion

6.1 Synthèse des performances

Les deux modèles sont calibrés sur le même dataset et évalués sur les mêmes options de validation.

Calibration sur les volatilités implicites :

La polynomiale mobilise 20 coefficients α_{ij} , tous calibrés sur les options, tandis que la famille exponentielle n'en calibre que 2 (β_0 et β_1), les paramètres de forme $\mu = 1$ et $\sigma_{hist} = 17,53\%$ étant fixés a priori sur données historiques. Cette différence de complexité se traduit directement dans les métriques : la polynomiale atteint un RMSE de 2,39 pts vol et une MAE de 1,22 pts vol, contre 6,41 pts vol et 5,20 pts vol pour la famille exponentielle. Les deux modèles garantissent une volatilité positive. La différence essentielle est que la polynomiale capture fidèlement le skew négatif et la structure par terme différenciée du SX5E, alors que la famille exponentielle, contrainte à une forme symétrique autour de ATM, produit une surface quasi-plate à 21% indépendamment du moneyness et de la maturité.

Pricing Monte-Carlo :

Critère	Polynomiale	Exponentielle
Biais ATM	10,21%	+2,82%
Biais puts ITM ($x \approx 0,90$)	+2,13%	+3,12%
Biais calls OTM ($x \approx 1,10$)	+40,64%	+73,95%

Sur la validation par pricing Monte-Carlo, le contraste est encore plus marqué. Dans la zone de validité des deux modèles (zone puts ITM), la polynomiale affiche un biais de +2.13% contre +3,12% pour l'exponentielle, les deux sont acceptables.

Mais dès ATM, la polynomiale présente un biais de +10,21% contre +22,82% pour l'exponentielle, et la divergence s'amplifie sur les calls OTM : +40,64% vs +73,95%.

Globalement, la MAE in-sample est de 19,4% pour la polynomiale contre 103,3% pour l'exponentielle. La famille exponentielle, réduite à une volatilité constante, surestime massivement la volatilité des calls OTM ($\sigma_{LV} \approx 21\%$ contre une IV observée de 14.5), ce qui gonfle mécaniquement tous les prix MC dès qu'on s'éloigne de ATM.

6.2 Avantages et limites de chaque modèle

Famille polynomiale

TABLE 7 – Comparaison des familles polynomiale et exponentielle

Avantages	
Famille polynomiale	Famille exponentielle
Flexibilité maximale : avec 20 coefficients bivariés en (x, t) , le polynôme peut approcher n'importe quelle surface régulière, y compris des asymétries complexes comme le skew négatif du SX5E.	Simplicité : 2 paramètres seulement sont calibrés. Pas de risque d'overfitting, les prix générés restent stables même avec des données peu nombreuses ou peu liquides.
RMSE le plus faible (2,39% pts vol) : meilleure reproduction des IV observées sur $[0, 60; 1, 40]$.	Interprétabilité totale : β_0 est la volatilité plancher, β_1 est l'amplitude du smile. Chaque paramètre a un sens économique immédiat.
Sélection rigoureuse du degré par AIC/BIC : le choix $n = 4, m = 3$ est justifié par des critères d'information, pas arbitraire.	Extrapolation stable : la gaussienne converge vers β_0 loin de ATM.
	Paramètres partiellement ancrés sur le marché réel : σ_{hist} est estimé sur les prix observés du SX5E, ce qui introduit une cohérence entre monde historique et monde risque-neutre.
Limites	
Famille polynomiale	Famille exponentielle
Extrapolation instable : au-delà de $x > 1, 10$ ou hors des maturités calibrées, σ_{LV} peut prendre des valeurs aberrantes, ce qui explique les biais élevés sur les calls OTM profonds.	Symétrie imposée : la gaussienne en $(x - 1)^2$ est symétrique autour de ATM. Or le skew du SX5E est structurellement asymétrique, ce qui conduit à $\beta_1 \approx 0$ et une surface plate.
Ininterprétable : les 20 coefficients α_{ij} n'ont pas de sens économique direct, on ne peut pas relier un paramètre à une propriété du marché.	σ_{hist} inadapté comme largeur du smile : la volatilité réalisée historique (17,53%) calibre la largeur de la cloche, mais la largeur du smile de volatilité implicite est gouvernée par les préférences de risque et la demande de couverture, pas par la volatilité réalisée.
Overfitting potentiel : avec 20 paramètres sur 4824 observations, le risque existe en théorie. En pratique, les performances out-of-sample restent comparables à celles in-sample, ce qui indique que le modèle généralise correctement.	RMSE élevé (6,41% pts vol) et pricing très dégradé dès que le smile est asymétrique.

6.3 Utilisation des modèles

Famille polynomiale

- Le dataset d'options est dense et liquide sur toute la surface (nombreuses maturités, strikes bien distribués), la flexibilité est exploitable et le risque d'overfitting est contrôlé.

- On veut pricer des options dans la zone de calibration (puts OTM à ITM modéré, maturités courtes à moyennes) avec la meilleure précision possible.
- Le sous-jacent présente un smile asymétrique prononcé que les formes paramétriques simples ne peuvent pas capturer.
- On dispose de ressources de calcul suffisantes pour re-calibrer fréquemment (la calibration est instantanée).

Famille exponentielle

- Les données sont peu nombreuses, 2 paramètres suffisent pour ne pas surapprendre un dataset de petite taille.
- L'interprétabilité est primordiale. Pour la communication avec des non-techniciens ou pour des rapports réglementaires, avoir deux paramètres économiquement lisibles est un avantage.
- Le smile du sous-jacent est approximativement symétrique (par exemple certaines matières premières ou indices moins skewés que les actions européennes).
- On souhaite une cohérence avec les données historiques via σ_{hist} .

6.4 Pistes pour réduire les biais

Pour la polynomiale, le principal levier est de restreindre le domaine de pricing à la zone de calibration ($x \in [0, 60; 1, 10]$) et d'utiliser la parité call-put pour les options hors zone plutôt que de faire confiance à l'extrapolation polynomiale.

Pour la famille exponentielle, la correction la plus directe serait de libérer μ et σ comme paramètre calibré plutôt que de le fixer à 1 et à la valeur historique du sous-jacent, cela permettrait au centre de la gaussienne de se déplacer vers les puts OTM ($\mu < 1$) pour capturer le pic de vol côté baissier, avec 4 paramètres calibrés au lieu de 2.

Annexes

A Code complet

Le code source complet du projet :

Listing 2 – Code Python

```
import numpy as np
import numpy.random as npr
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import brentq
from scipy.optimize import minimize
import warnings
warnings.filterwarnings('ignore')
from scipy.optimize import differential_evolution
from scipy.optimize import minimize_scalar
from scipy.optimize import least_squares
import numpy.random as npr
from scipy.optimize import brentq
from scipy.stats import norm
import yfinance as yf

def parse_cir_data(filepath: str, skip_row=0) -> pd.DataFrame:
    df = pd.read_csv(filepath, sep=',', skiprows=skip_row, header=0)
    df.columns = ['date', 'time_period', 'estr']
    df['date'] = pd.to_datetime(df['date'])
    df['estr_pct'] = df['estr'] / 100
    df_fit = df[df['date'] >= '2023-01-01'].copy().reset_index(drop=True)
    r = df_fit['estr_pct'].values
    dt = 1 / 252 # pas journalier

    r_t = r[:-1] # r_t
    r_tp1 = r[1:] # r_{t+dt}

    return df, r_t, r_tp1, dt

def neg_log_likelihood_cir(params, r_t, r_tp1, dt):
    """
    Log-vraisemblance ngative du CIR discrtis.
    On modlise  $dr \sim N(\text{drift}, \text{variance})$  o :
        drift =  $a \cdot (b - r_t) \cdot dt$ 
        variance =  $\gamma \cdot r_t \cdot dt$ 
    """
    a, b, gamma = params

    # Drift et variance attendus
    drift = a * (b - r_t) * dt
    variance = gamma**2 * np.abs(r_t) * dt

    # dr observ
    dr = r_tp1 - r_t

    # Log-vraisemblance d'une gaussienne  $N(\text{drift}, \text{variance})$ 
    #  $\log p(dr) = -0.5 \cdot \log(2 \cdot \text{var}) - (dr - \text{drift})^2 / (2 \cdot \text{var})$  : densit de probabilit d'une
    # gaussienne
    log_p = (- 0.5 * np.log(2 * np.pi * variance)
             - (dr - drift)**2 / (2 * variance))
```



```

# we take negative because the optimizer minimizes the function
return -np.sum(log_p)

def get_params_cir(cir_data_path: str, skip_row=0):
    """
    Estimate the parameters of the CIR model using maximum likelihood estimation.
    """
    df, r_t, r_tp1, dt = parse_cir_data(cir_data_path, skip_row=skip_row)
    bounds = [
        (0.01, 10.0), # a : between 0.01 and 10
        (0.001, 0.10), # b : between 0.1% and 10%
        (0.001, 0.50), # gamma : between 0.1% and 50%
    ]

    res = differential_evolution(
        neg_log_likelihood_cir,
        bounds=bounds,
        args = (r_t, r_tp1, dt),
        seed = 42, # reproducible
        maxiter = 1000,
        tol = 1e-7,
    )

    return res.x # a, b, gamma

def parse_option_data(
    filepath: str,
    skip_row=0,
    date_ref=None,
    spot_price=6071.4,
    x_col='moneyness',
) -> pd.DataFrame:
    """
    Parse the option data from a CSV file.
    """
    df = pd.read_csv(filepath, sep=';', skiprows=skip_row, header=0)
    print(f"Data imported from {filepath}. Number of rows in raw data: {len(df)}")
    # print(f"Columns: {df.columns.tolist()}")
    df.columns = ['ticker', 'expire_date', 'strike', 'pct_moneyness', 'price', 'strike_pct', '
    type']

    # get time to maturity in years
    date_ref = pd.to_datetime(date_ref, dayfirst=True)
    df['ref_date'] = date_ref
    df['T'] = (pd.to_datetime(df['expire_date'], dayfirst=True) - date_ref).dt.days / 365

    # Convert price to float and replace commas with dots
    df['strike'] = df['strike'].apply(lambda x: float(str(x).replace(',', '.)))
    df['price'] = df['price'].apply(lambda x: float(str(x).replace(',', '.)))

    # Recalculate pct_moneyness based on strike and spot price provided
    df[x_col] = (df['strike'] / spot_price)

    # Keep only relevant lines
    # Filter only on ticker containing 'SX5E' and type in ['C', 'P']
    df = df[df['ticker'].str.contains('SX5E')]
    df = df[df['type'].isin(['C', 'P'])]
    df.dropna(subset=['T', 'strike', 'price'], inplace=True)

    df = df[['ticker', 'T', 'strike', x_col, 'price', 'type']]

    return df

```

```

def bs_price(S, K, T, r, sigma, opt_type):
    """
    Prix Black-Scholes analytique
     $S_t = S_0 \cdot \exp(\sigma \cdot B_t + (\nu - \sigma^2 / 2)t)$ .
    """
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    if opt_type == 'C':
        return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    else:
        return K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)

def implied_vol(price, S, K, T, r, opt_type):
    result = minimize_scalar(
        lambda s: (bs_price(S, K, T, r, s, opt_type) - price)**2,
        bounds = (1e-4, 5),
        method = 'bounded',
        options = {'xatol': 1e-6}
    )
    iv = result.x
    return iv if 0 < iv < 10.0 else np.nan

def plot_iv_surface(df_processed):
    fig, ax = plt.subplots(figsize=(9, 5))
    fig.suptitle(
        "Surface de volatilit implicite BS observe SX5E (09/06/2026)",
        fontsize=13, fontweight='bold'
    )
    mats_plot = [0.277, 0.449, 0.698, 1.024, 1.522]
    colors = plt.cm.plasma(np.linspace(0.1, 0.9, len(mats_plot)))
    for T_sel, col in zip(mats_plot, colors):
        sub = df_processed[np.abs(df_processed['T'] - T_sel) < 0.03].sort_values('moneyness')
        if len(sub) >= 4:
            ax.plot(sub['moneyness'], sub['implied_vol'] * 100,
                    color=col, lw=2, marker='o', ms=3,
                    label=f'T = {T_sel:.2f} a')
    ax.axvline(1.0, color='k', lw=1, ls='--')
    ax.set_xlabel('Moneyness K / S0')
    ax.set_ylabel('Vol implicite (%)')
    ax.set_title('Smile par maturit')
    ax.legend(fontsize=9)
    ax.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig('fig/fig1_surface_IV.png', dpi=150, bbox_inches='tight')
    plt.show()

# =====
# STATISTIQUES DESCRIPTIVES DE LA SURFACE IV
# Skew, structure par terme, analyse par zone de moneyness
# =====

def description_statistics(df, x_col='moneyness', y_col='implied_vol'):
    print("\n" + "="*60)
    print("STATISTIQUES DESCRIPTIVES - SURFACE IV OBSERVE")
    print("="*60)

    print(f" IV range : {df[y_col].min():.1%} - {df[y_col].max():.1%}")
    atm = df[df[x_col].between(0.97, 1.03)][y_col].mean()
    print(f" IV ATM moyenne : {atm:.1%}")

    # 1. Skew par zone de moneyness
    print("\n Skew (IV moyenne par zone de moneyness) ")
    zones = [

```

```

("Deep OTM   (K/S < 0.80)", df[x_col] < 0.80),
("Puts OTM   (K/S [0.80, 0.95])", df[x_col].between(0.80, 0.95)),
("ATM        (K/S [0.97, 1.03])", df[x_col].between(0.97, 1.03)),
("Calls OTM  (K/S [1.05, 1.20])", df[x_col].between(1.05, 1.20)),
("Deep OTM   (K/S > 1.20)", df[x_col] > 1.20),
]

for label, mask in zones:
    sub = df[mask]
    calls = sub[sub['type'] == 'C']
    puts = sub[sub['type'] == 'P']
    iv_c = f"{calls[y_col].mean():.1%} (N={len(calls)})" if len(calls) > 0 else ""
    iv_p = f"{puts[y_col].mean():.1%} (N={len(puts)})" if len(puts) > 0 else ""
    iv_t = f"{sub[y_col].mean():.1%} (N={len(sub)})"
    print(f" {label:<35} {iv_c:>20} {iv_p:>20} {iv_t:>15}")

# 2. Structure par terme ATM
print("\n Structure par terme ATM (K/S [0.97, 1.03]) ")
print(f" {'T':>8} {'Calls':>15} {'Puts':>15} {'Total':>12}")
mats_ref = [0.10, 0.28, 0.45, 0.70, 1.02, 1.52, 1.77]
tol = 0.04
for T_sel in mats_ref:
    sub = df[(np.abs(df['T'] - T_sel) < tol) & df[x_col].between(0.97, 1.03)]
    calls = sub[sub['type'] == 'C']
    puts = sub[sub['type'] == 'P']
    if len(sub) >= 3:
        iv_c = f"{calls[y_col].mean():.1%} (N={len(calls)})" if len(calls) > 0 else ""
        iv_p = f"{puts[y_col].mean():.1%} (N={len(puts)})" if len(puts) > 0 else ""
        print(f" T {T_sel:.2f}a {iv_c:>20} {iv_p:>20} {sub[y_col].mean():.1%} (N={len(sub)})")

# 3. Structure par terme Puts OTM (K/S 0.85)
print("\n Structure par terme Puts OTM (K/S [0.82, 0.88]) ")
print(f" {'T':>8} {'Calls':>15} {'Puts':>15} {'Total':>12}")
for T_sel in mats_ref:
    sub = df[(np.abs(df['T'] - T_sel) < tol) & df[x_col].between(0.82, 0.88)]
    calls = sub[sub['type'] == 'C']
    puts = sub[sub['type'] == 'P']
    if len(sub) >= 3:
        iv_c = f"{calls[y_col].mean():.1%} (N={len(calls)})" if len(calls) > 0 else ""
        iv_p = f"{puts[y_col].mean():.1%} (N={len(puts)})" if len(puts) > 0 else ""
        print(f" T {T_sel:.2f}a {iv_c:>20} {iv_p:>20} {sub[y_col].mean():.1%} (N={len(sub)})")

# 4. Structure par terme Calls OTM (K/S 1.10)
print("\n Structure par terme Calls OTM (K/S [1.07, 1.13]) ")
print(f" {'T':>8} {'Calls':>15} {'Puts':>15} {'Total':>12}")
for T_sel in mats_ref:
    sub = df[(np.abs(df['T'] - T_sel) < tol) & df[x_col].between(1.07, 1.13)]
    calls = sub[sub['type'] == 'C']
    puts = sub[sub['type'] == 'P']
    if len(sub) >= 3:
        iv_c = f"{calls[y_col].mean():.1%} (N={len(calls)})" if len(calls) > 0 else ""
        iv_p = f"{puts[y_col].mean():.1%} (N={len(puts)})" if len(puts) > 0 else ""
        print(f" T {T_sel:.2f}a {iv_c:>20} {iv_p:>20} {sub[y_col].mean():.1%} (N={len(sub)})")

# 5. Rsum global
print("\n Rsum global ")
calls_all = df[df['type'] == 'C']
puts_all = df[df['type'] == 'P']
print(f" N total : {len(df)} (calls={len(calls_all)}, puts={len(puts_all)})")
print(f" IV min / max : {df[y_col].min():.1%} / {df[y_col].max():.1%}")

```

```

print(f" IV ATM moyenne      : {df[df[x_col].between(0.97,1.03)][y_col].mean():.1%}")
print(f" Skew puts OTM - ATM : "
      f"{(df[df[x_col]<0.90][y_col].mean() - df[df[x_col].between(0.97,1.03)][y_col].mean())
*100:+.1f} pts vol")
print(f" Calls OTM - ATM      : "
      f"{(df[df[x_col]>1.05][y_col].mean() - df[df[x_col].between(0.97,1.03)][y_col].mean())
*100:+.1f} pts vol")

# =====
# Famille de modles polynomiaux
# =====

def build_design_matrix(x_arr, t_arr, n_deg, m_deg):
    """
    Construit la matrice de design X de taille (N, (n+1)*(m+1)).
    Colonne (i,j) = x^i * t^j pour i dans 0..n, j dans 0..m.
    """
    cols = [
        (x_arr ** i) * (t_arr ** j)
        for i in range(n_deg + 1)
        for j in range(m_deg + 1)
    ]
    return np.column_stack(cols)

def calibrer_polynomiale(df_in, n_deg, m_deg, lam=1e-4, x_col='moneyness', y_col='implied_vol')
:
    """
    Calibration Ridge par least_squares.
    Utilisation de matrice de design X construite partir des polynmes en x et t.
    On minimise la fonction de cot :
        ||X @ alpha - y||**2 + lam * ||alpha||**2
    """
    X = build_design_matrix(df_in[x_col].values, df_in['T'].values, n_deg, m_deg)
    y = df_in[y_col].values

    def residus_polynomial(alpha):
        """
        Vecteur de rsidus assembl :
        - partie donnees : X @ alpha - y
        - partie Ridge    : sqrt(lam) * alpha
        least_squares minimise la somme des carrs de ce vecteur
        = ||X @ alpha - y||**2 + lam * ||alpha||**2
        """
        residus_fit = X @ alpha - y
        residus_reg = np.sqrt(lam) * alpha
        return np.concatenate([residus_fit, residus_reg])

    # Point de dpart : zros
    # (Ridge est convexe x0 n'influe pas sur le rsultat)
    x0 = np.zeros(X.shape[1])

    res = least_squares(
        residus_polynomial,
        x0,
        method = 'lm',          # Levenberg-Marquardt
        ftol    = 1e-12,
        xtol    = 1e-12,
    )

    alpha_hat = res.x
    iv_hat    = X @ alpha_hat
    rmse      = np.sqrt(np.mean((y - iv_hat)**2))
    mae       = np.mean(np.abs(y - iv_hat))

```

```

frac_neg = np.mean(iv_hat < 0)

return alpha_hat, rmse, mae, frac_neg, iv_hat

def results_polynomial_selection(df, n_max=4, m_max=3, lam=1e-4, x_col='moneyness', y_col='
implied_vol'):
    """
    Teste toutes les combinaisons (n, m) avec n dans 1..n_max, m dans 1..m_max.
    Retient le degr minimisant AIC et BIC :
        AIC = N * log(RSS/N) + 2*k
        BIC = N * log(RSS/N) + log(N)*k
    avec k = (n+1)*(m+1) nombre de paramtres.
    """
    print("\n== Slection du degr polynomial (n, m) ==")

    results = []
    N = len(df)

    for n in range(1, n_max + 1):
        for m in range(1, m_max + 1):
            a_hat, rmse, mae, fn, yhat = calibrer_polynomiale(df, n, m, lam, x_col=x_col, y_col=
y_col)
            k = (n + 1) * (m + 1)
            RSS = np.sum((df[y_col].values - yhat) ** 2)

            # AIC : pnalise chaque paramtre par 2
            aic = N * np.log(RSS / N) + 2 * k

            # BIC : pnalise chaque paramtre par log(N)
            # log(2919) 7.98 pnalit ~4x plus forte qu'AIC
            bic = N * np.log(RSS / N) + np.log(N) * k

            score = rmse + 0.10 * fn
            results.append({
                'n': n, 'm': m, 'k': k,
                'rmse': rmse, 'mae': mae, 'frac_neg': fn,
                'alpha_hat': a_hat, 'score': score,
                'aic': aic, 'bic': bic          #- bic ajout
            })

    # Save results to CSV
    results_df = pd.DataFrame(results)
    results_df.to_csv('data/results_polynomial_selection.csv', index=False)
    # Best according to AIC
    best_aic = min(results, key=lambda r: r['aic'])
    # Best according to BIC
    best_bic = min(results, key=lambda r: r['bic'])

    print(f"\n Best AIC : n={best_aic['n']}, m={best_aic['m']} "
          f"({best_aic['k']} parameters) AIC={best_aic['aic']:.1f}")
    print(f"\n Best BIC : n={best_bic['n']}, m={best_bic['m']} "
          f"({best_bic['k']} parameters) BIC={best_bic['bic']:.1f}")

    # Display coefficients of the best AIC
    n_opt = best_aic['n']
    m_opt = best_aic['m']
    print(f"\n RMSE = {best_aic['rmse']*100:.3f} vol points (AIC optimal)")
    print(f"\n MAE = {best_aic['mae']*100:.3f} vol points")
    print(f"\n % calibrated vol < 0 = {best_aic['frac_neg']:.2%}")

    print(f"\n Coefficients sigma_ij (AIC optimal model) :")
    for idx, (i, j) in enumerate([(i, j)
        for i in range(n_opt + 1)
        for j in range(m_opt + 1)]):

```

```

        print(f"      sigma_{i}{j}  (x^{i}t^{j}) = {best_aic['alpha_hat'][idx]:+.6f}")

    # Display coefficients of the best BIC
    n_opt = best_bic['n']
    m_opt = best_bic['m']
    print(f"\n  RMSE = {best_bic['rmse']*100:.3f} vol points  (BIC optimal)")
    print(f"  MAE  = {best_bic['mae']*100:.3f} vol points")
    print(f"  % calibrated vol < 0 = {best_bic['frac_neg']:.2%}")

    print(f"\n  Coefficients sigma_ij (BIC optimal model) :")
    for idx, (i, j) in enumerate([(i, j)
                                   for i in range(n_opt + 1)
                                   for j in range(m_opt + 1)]):
        print(f"      sigma_{i}{j}  (x^{i}t^{j}) = {best_bic['alpha_hat'][idx]:+.6f}")

    # Return the two best models
    return results, best_aic, best_bic

def sigma_poly(t, x_norm, alpha, n, m):
    """
    sigma_LV(t, x_norm) avec x_norm = K/S0.
    Signature aligne avec params_model['sigma_poly'] : alpha, n, m.
    Clip 0.1% pour viter les vol ngatives hors chantillon.
    """
    feats = np.array([
        (x_norm ** i) * (t ** j)
        for i in range(n + 1)
        for j in range(m + 1)
    ])
    return max(float(np.dot(feats, alpha)), 0.001)

def plot_calibration_polynomial(df, N_OPT, M_OPT, ALPHA_OPT, x_col='moneyness', y_col='
    implied_vol'):
    """
    Plot the calibration results.
    Surface of Smile observed.
    """

    # Prediction of the implied volatility using the polynomial model
    X_all = build_design_matrix(df[x_col].values, df['T'].values, N_OPT, M_OPT)
    iv_hat = X_all @ ALPHA_OPT
    df = df.copy()
    df['IV_fit'] = iv_hat
    df['residu'] = (df[y_col] - df['IV_fit']) * 100

    rmse_val = np.sqrt(np.mean((df[y_col] - df['IV_fit'])**2)) * 100
    mae_val = np.mean(np.abs(df[y_col] - df['IV_fit'])) * 100

    fig, ax = plt.subplots(figsize=(9, 5))
    mats = [0.277, 0.449, 0.698, 1.024, 1.522]
    cols = plt.cm.plasma(np.linspace(0.1, 0.9, len(mats)))
    for T_sel, col in zip(mats, cols):
        sub = df[np.abs(df['T'] - T_sel) < 0.03].sort_values(x_col)
        if len(sub) < 4:
            continue
        ax.scatter(sub[x_col], sub[y_col] * 100,
                   color=col, s=12, zorder=3, alpha=0.8)
        x_sm = np.linspace(sub[x_col].min(), sub[x_col].max(), 100)
        y_sm = [sigma_poly(T_sel, xi, ALPHA_OPT, N_OPT, M_OPT) * 100 for xi in x_sm]
        ax.plot(x_sm, y_sm, color=col, lw=2, label=f'T={T_sel:.2f}a')
    ax.axvline(1.0, color='k', lw=1, ls='--')
    ax.set_xlabel('Moneyness K/S0')
    ax.set_ylabel('Vol (%)')

```

```

ax.set_title('Smile : IV observe (pts) vs sigma_LV (courbe)')
ax.legend(fontsize=8)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('fig/fig2_polynomial_calibration.png', dpi=150, bbox_inches='tight')
plt.show()

# =====
# Famille de modes exponentiels
# =====

def get_param_hist_sx5e():
    ticker = yf.Ticker("^STOXX50E")
    df_hist = ticker.history(period="5y")
    df_hist = df_hist.sort_index()
    df_hist['log_ret'] = np.log(df_hist['Close'] / df_hist['Close'].shift(1))
    df_hist = df_hist.dropna(subset=['log_ret'])

    # Yearly drift and volatility
    MU_HIST = df_hist['log_ret'].mean() * 252 # drift
    SIGMA_HIST = df_hist['log_ret'].std() * np.sqrt(252) # vol

    print(f"Drift historique (annuel) : {MU_HIST:.4f}")
    print(f"Volatilit historique (annuelle) : {SIGMA_HIST:.4f}")

    return MU_HIST, SIGMA_HIST

def sigma_lv_exp(x_norm, t, beta0, beta1, sigma_hist, eps_hist):
    """
    Famille exponentielle consigne stricte.

    sigma_LV(t, x) = beta0 + beta1 * exp(-(x - 1)^2 / (2 * sigma^2 * (t + eps)))

    Paramtres
    -----
    x_norm : moneyness K/S0 (mu=1 = ATM, fix)
    t       : maturit en annees
    beta0   : vol plancher (calibr)
    beta1   : amplitude smile (calibr)
    sigma_hist : vol historique annualise (FIXE = sigma_hist)
    eps_hist : buffer t0 (FIXE = 1 semaine)

    Pourquoi x - 1 et non log(x) ?
    La consigne exprime la dpendance en (S_t - mu) en niveaux de prix.
    En moneyness normalis x = K/S0, cela devient (x - 1), centr en ATM.
    La gaussienne rsultante est symtrique en moneyness autour de ATM.
    """
    exposant = np.clip(
        -(x_norm - 1.0)**2 / (2 * sigma_hist**2 * (t + eps_hist)),
        -500, 0
    )
    return np.clip(beta0 + beta1 * np.exp(exposant), 0.001, 5.0)

def sigma_svi_safe(t, x_norm, beta0, beta1, sigma_hist, eps_hist):
    """Interface identique sigma_poly utilisable dans le pricer MC."""
    return float(sigma_lv_exp(float(x_norm), float(t), beta0, beta1, sigma_hist, eps_hist))

def objectif_exp(params, x_arr, t_arr, iv_arr, sigma_hist, eps_hist):
    """MCO : min_{beta0,beta1} sum (sigma_lv_exp - IV)^2"""
    beta0, beta1 = params
    if beta0 <= 0 or beta1 <= 0 or beta0 > 2.0 or beta1 > 2.0:
        return 1e10
    iv_hat = sigma_lv_exp(x_arr, t_arr, beta0, beta1, sigma_hist, eps_hist)

```

```

    return float(np.sum((iv_hat - iv_arr)**2))

def calibrer_exp(df, eps, sigma_hist, x_col='moneyness', y_col='implied_vol'):
    x_arr = df[x_col].values
    t_arr = df['T'].values
    iv_arr = df[y_col].values

    res = differential_evolution(
        objectif_exp,
        bounds = [(0.05, 0.80), (0.001, 0.50)],
        args = (x_arr, t_arr, iv_arr, sigma_hist, eps),
        seed = 42,
        maxiter = 3000,
        tol = 1e-10,
        popsize = 20,
        mutation = (0.5, 1.5),
        recombination = 0.9,
    )

    beta0_opt, beta1_opt = res.x
    iv_hat = sigma_lv_exp(x_arr, t_arr, beta0_opt, beta1_opt, sigma_hist, eps)
    rmse = float(np.sqrt(np.mean((iv_arr - iv_hat)**2)))
    mae = float(np.mean(np.abs(iv_arr - iv_hat)))

    print("\nCalibration famille exponentielle")
    print(f"\n=== Paramtres calibrs ===")
    print(f" [FIXE] sigma_hist = {sigma_hist:.4f} ({sigma_hist*100:.2f}%) - vol ralise historique")
    print(f" [FIXE] mu = 1.0000 - centre ATM (K/S = 1)")
    print(f" [CALIB] beta0 = {beta0_opt:.4f} ({beta0_opt*100:.2f}%) - vol plancher")
    print(f" [CALIB] beta1 = {beta1_opt:.4f} ({beta1_opt*100:.2f}%) - amplitude smile")
    print(f"\n RMSE = {rmse*100:.3f} pts de vol")
    print(f" MAE = {mae*100:.3f} pts de vol")
    print(f" % vol < 0 = 0.00% (garanti car beta0 > 0)")

    return beta0_opt, beta1_opt, rmse, mae, iv_hat

def plot_calibration_exp(df, beta0, beta1, sigma_hist, eps, x_col='moneyness', y_col='implied_vol', lower_x = 0.60, upper_x = 1.40):
    """
    Smile observ (points) vs calibr (courbe) par maturit
    """
    # Prdictions sur le dataset
    df = df.copy()
    df = df[df[x_col].between(lower_x, upper_x)]
    df = df[(df[y_col] > 0.05) & (df[y_col] < 1.50)].reset_index(drop=True)
    iv_hat = sigma_lv_exp(df[x_col].values, df['T'].values, beta0, beta1, sigma_hist, eps)
    df['IV_fit'] = iv_hat
    df['residu'] = (df[y_col] - df['IV_fit']) * 100
    print(len(df), sigma_hist, eps)

    rmse_val = np.sqrt(np.mean((df[y_col] - df['IV_fit'])**2)) * 100
    mae_val = np.mean(np.abs(df[y_col] - df['IV_fit'])) * 100

    # Grille pour le contour
    x_g = np.linspace(0.60, 1.40, 100)
    t_g = np.linspace(0.08, 1.80, 100)
    XX, TT = np.meshgrid(x_g, t_g)
    ZZ = sigma_lv_exp(XX, TT, beta0, beta1, sigma_hist, eps) * 100

    mats = [0.277, 0.449, 0.698, 1.024, 1.522]
    cols = plt.cm.plasma(np.linspace(0.1, 0.9, len(mats)))

```



```

fig, ax = plt.subplots(figsize=(9, 5))
fig.suptitle(
    f"Calibration famille exponentielle (2 paramtres calibrs)\n"
    f"={beta0*100:.2f}% =={beta1*100:.2f}% "
    f"sigma_hist={sigma_hist*100:.2f}% (fixe) =1 (fixe) | "
    f"RMSE={rmse_val:.2f} pts vol MAE={mae_val:.2f} pts vol",
    fontsize=11, fontweight='bold'
)

# Graphe 3 : smile observ vs calibr par maturit
for T_sel, col in zip(mats, cols):
    sub = df[np.abs(df['T'] - T_sel) < 0.03].sort_values('moneyness')
    if len(sub) < 4:
        continue
    ax.scatter(sub['moneyness'], sub[y_col] * 100,
               color=col, s=12, zorder=3, alpha=0.8)
    x_sm = np.linspace(sub['moneyness'].min(), sub['moneyness'].max(), 100)
    y_sm = sigma_lv_exp(x_sm, T_sel, beta0, beta1, sigma_hist, eps) * 100
    ax.plot(x_sm, y_sm, color=col, lw=2, label=f'T={T_sel:.2f}a')

ax.axvline(1.0, color='k', lw=1, ls='--')
ax.set_xlabel('Moneyness K/S')
ax.set_ylabel('Vol (%)')
ax.set_title('Smile : IV observe (pts) vs _LV exp (courbe)')
ax.legend(fontsize=8)
ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('fig/fig3_calibration_exp.png', dpi=150, bbox_inches='tight')
plt.show()

# =====
# PRICER MONTE-CARLO SOUS Q (Euler discretization)
# 2 versions : sigma_poly (polynomial) and sigma_svi_safe (exponential)
# =====

def simulate_paths(
    T_mat,
    model: str,
    alpha_cir: float,
    beta_cir: float,
    gamma_cir: float,
    r0: float,
    Spot: float,
    parameters_model: dict,
    sigma_hist=None,
    eps_hist=None,
    n_steps=200, N_sim=4000, seed=0
):
    """
    Simule N_sim trajectoires for (r_t, S_t) on [0, T_mat].

    Additional parameters:
        model : str
            'poly' - uses sigma_poly (polynomial family)
            'svi' - uses sigma_svi (exponential family)
        alpha_cir, beta_cir, gamma_cir : float
            CIR parameters for the short rate process
        r0 : float
            Initial short rate
        Spot : float
            Initial spot price
        n_steps : int
    """

```

```

        Number of time steps in the simulation. Default is 200.
    N_sim : int
        Number of Monte Carlo simulations. Default is 4000.

    Returns
    -----
    S_T      : array (N_sim,) - final price S_T = Spot * ST * S0_T
    moy_S    : array (N_sim,) - rolling average (1/T) S du
    disc     : array (N_sim,) - discount factor  $e^{-r \text{ du}}$ 
    """
    p      = T_mat / n_steps
    rp     = np.sqrt(p)
    rng     = npr.default_rng(seed)

    r      = r0 * np.ones(N_sim)
    ST     = np.ones(N_sim)
    int_r  = np.zeros(N_sim)
    sum_S  = np.zeros(N_sim)

    # Choix de la fonction de vol locale selon le modle
    if model == 'poly':
        params = parameters_model['sigma_poly']
        fn_sigma = lambda t, s: sigma_poly(t, s, params['alpha'], params['n'], params['m'])
    elif model == 'svi':
        if sigma_hist is None or eps_hist is None:
            raise ValueError("sigma_hist et eps_hist sont requis pour le modle 'svi'.")
        params = parameters_model['sigma_svi']
        fn_sigma = lambda t, s: sigma_svi_safe(
            t, s, params['beta0'], params['beta1'], sigma_hist, eps_hist
        )
    else:
        raise ValueError("Le parametre 'model' doit tre 'poly' ou 'svi'.")

    for i in range(1, n_steps + 1):
        t_prev = (i - 1) * p
        Gi = rng.standard_normal(N_sim)
        Hi = rng.standard_normal(N_sim)

        # Taux CIR cours p.21
        r = np.clip(
            r + alpha_cir * (beta_cir - r) * p
            + gamma_cir * np.sqrt(np.abs(r)) * rp * Gi,
            0.0, 0.30
        )
        int_r += r * p

        # Local vol
        ST_clipped = np.clip(ST, 1e-4, 10.0)
        sig = np.vectorize(lambda s: fn_sigma(t_prev, s))(ST_clipped)
        sig = np.clip(sig, 0.001, 5.0)

        ST      = np.clip(ST + sig * ST * rp * Hi, 1e-8, 1e6)
        S_t     = Spot * ST * np.exp(np.minimum(int_r, 20))
        sum_S += S_t * p

    disc = np.exp(-np.minimum(int_r, 20))
    S_T  = Spot * ST * np.exp(np.minimum(int_r, 20))
    moy_S = sum_S / T_mat

    return S_T, moy_S, disc

def pricer_call_MC(
    T_mat, K,

```

```

alpha_cir, beta_cir, gamma_cir, r0, Spot,
parameters_model,
sigma_hist=None,
eps_hist=None,
n_steps=200,
N_sim=4000, seed=0, modele='poly'):
    """
    Prix d'un call européen par Monte-Carlo.
    modele : 'poly' ou 'svi'
    """
    S_T, _, disc = simulate_paths(
        T_mat, modele, alpha_cir, beta_cir, gamma_cir, r0, Spot,
        parameters_model, sigma_hist=sigma_hist, eps_hist=eps_hist,
        n_steps=n_steps, N_sim=N_sim, seed=seed
    )
    return float(np.mean(np.maximum(S_T - K, 0) * disc))

def selectionner_options_test(df, maturites, pcts_moneyness, tol=0.03):
    """Sélectionne les options les plus proches des (T, K/S0) cibles."""
    rows = []
    for T_sel in maturites:
        sub = df[(np.abs(df['T'] - T_sel) < tol) & (df['type'] == 'C')]
        for pct in pcts_moneyness:
            closest = sub.iloc[(sub['moneyness'] - pct).abs().argsort()[:1]]
            if len(closest):
                rows.append(closest.iloc[0])
    return (pd.DataFrame(rows)
            .drop_duplicates(subset=['T', 'strike'])
            .reset_index(drop=True))

def valider_pricing(
    df_sel,
    label,
    alpha_cir,
    beta_cir,
    gamma_cir,
    r0, Spot,
    parameters_model,
    sigma_hist=None,
    eps_hist=None,
    n_steps=200,
    N_sim=4000,
    seed=42
):
    """
    Compare prix MC (poly ET svi) vs prix march.
    Affiche un tableau avec les deux modèles cte cte.
    Retourne deux DataFrames : df_poly et df_svi.
    """
    print(f"\n{' '*90}")
    print(f" {label}")
    print(f"{' '*90}")
    print(f"{'T':>7} {'K':>6} {'P_obs':>8} "
          f"{'P_MC Poly':>10} {'Err% Poly':>10} "
          f"{'P_MC SVI':>10} {'Err% SVI':>10} "
          f"{'IV_obs':>8}")
    print(f"{' '*90}")

    records_poly, records_svi = [], []
    poly_params = parameters_model['sigma_poly']
    svi_params = parameters_model['sigma_svi']

```

```

for _, row in df_sel.iterrows():
    T_r = float(row['T'])
    K_r = float(row['strike'])
    P_obs = float(row['price'])
    IV_obs = float(row['implied_vol'])

    # Pricer avec les deux modes
    P_poly = pricer_call_MC(
        T_r, K_r, alpha_cir, beta_cir, gamma_cir, r0, Spot,
        parameters_model, n_steps=n_steps, N_sim=N_sim, seed=seed, modele='poly'
    )
    P_svi = pricer_call_MC(
        T_r, K_r, alpha_cir, beta_cir, gamma_cir, r0, Spot,
        parameters_model, sigma_hist=sigma_hist, eps_hist=eps_hist,
        n_steps=n_steps, N_sim=N_sim, seed=seed, modele='svi'
    )

    err_poly = (P_poly - P_obs) / P_obs * 100
    err_svi = (P_svi - P_obs) / P_obs * 100

    records_poly.append({
        'T': T_r, 'K': K_r, 'P_obs': P_obs, 'P_MC': P_poly,
        'err_abs': P_poly-P_obs, 'err_pct': err_poly,
        'IV_obs': IV_obs,
        'sig_lv': sigma_poly(T_r, K_r/Spot, poly_params['alpha'], poly_params['n'],
poly_params['m'])
    })
    records_svi.append({
        'T': T_r, 'K': K_r, 'P_obs': P_obs, 'P_MC': P_svi,
        'err_abs': P_svi-P_obs, 'err_pct': err_svi,
        'IV_obs': IV_obs,
        'sig_lv': sigma_svi_safe(
            T_r, K_r/Spot, svi_params['beta0'], svi_params['beta1'], sigma_hist, eps_hist
        )
    })

print(f"{T_r:>7.3f} {K_r:>6.0f} {P_obs:>8.1f} "
      f"{P_poly:>10.1f} {err_poly:>+10.1f}% "
      f"{P_svi:>10.1f} {err_svi:>+10.1f}% "
      f"{IV_obs:>8.1f}%")

df_poly = pd.DataFrame(records_poly)
df_svi = pd.DataFrame(records_svi)

df_poly.to_csv('data/results_pricing_poly.csv', index=False)
df_svi.to_csv('data/results_pricing_svi.csv', index=False)

# Mtriques sur les options liquides (prix > 200 pts)
for name, df_r in [('Polynomiale', df_poly), ('SVI', df_svi)]:
    liq = df_r[df_r['P_obs'] > 200]
    mae = np.mean(np.abs(liq['err_pct']))
    rmse = np.sqrt(np.mean(liq['err_pct']**2))
    bias = np.mean(liq['err_pct'])
    print(f"\n [{name}] (calls > 200 pts, N={len(liq)}) "
          f"MAE={mae:.1f}% RMSE={rmse:.1f}% Biais={bias:+.1f}%")

return df_poly, df_svi

def plot_pricing_results(df_in_poly, df_in_svi, df_out_poly, df_out_svi):
    """
    Figure comparaison pricing :
    Ligne 1 : P_obs vs P_MC (in-sample poly | in-sample svi | out poly)
    Ligne 2 : Biais poly vs SVI par maturit
    """

```

```

"""
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle(
    "Validation pricing Monte-Carlo - Polynomiale vs SVI exponentielle",
    fontsize=13, fontweight='bold'
)

# Ligne 1 : scatter P_obs vs P_MC
configs = [
    (axes[0,0], df_in_poly, 'In-sample - Polynomiale', 'viridis'),
    (axes[0,1], df_in_svi, 'In-sample - SVI', 'plasma'),
    (axes[0,2], df_out_poly, 'Out-of-sample - Polynomiale', 'cividis'),
]
for ax, df_r, label, cmap in configs:
    sub = df_r[df_r['P_obs'] > 200]
    if len(sub) == 0:
        continue
    sc = ax.scatter(sub['P_obs'], sub['P_MC'], c=sub['T'],
                    cmap=cmap, s=80, zorder=3, edgecolors='k', lw=0.5)
    lim = max(sub['P_obs'].max(), sub['P_MC'].max()) * 1.05
    ax.plot([0, lim], [0, lim], 'k--', lw=1.5, label='Fit parfait')
    plt.colorbar(sc, ax=ax, label='T (ans)')
    mae = np.mean(np.abs((sub['P_MC'] - sub['P_obs']) / sub['P_obs'])) * 100
    ax.set(xlabel='Prix march (pts)', ylabel='Prix MC (pts)',
           title=f'{label}\nMAE = {mae:.1f}% N={len(sub)}')
    ax.legend(fontsize=9)
    ax.grid(alpha=0.3)

# Ligne 2 : biais par maturit poly vs svi
# In-sample
all_in_poly = df_in_poly[df_in_poly['P_obs'] > 200]
all_in_svi = df_in_svi[df_in_svi['P_obs'] > 200]
mats_in = sorted(all_in_poly['T'].round(3).unique())
bias_in_poly, bias_in_svi = [], []
for T_sel in mats_in:
    s_p = all_in_poly[np.abs(all_in_poly['T']-T_sel)<0.015]
    s_s = all_in_svi[np.abs(all_in_svi['T']-T_sel)<0.015]
    bias_in_poly.append(np.mean(s_p['err_pct']) if len(s_p) else 0)
    bias_in_svi.append(np.mean(s_s['err_pct']) if len(s_s) else 0)

x_p = np.arange(len(mats_in))
w = 0.38
axes[1,0].bar(x_p - w/2, bias_in_poly, width=w, color='steelblue',
              alpha=0.8, label='Polynomiale')
axes[1,0].bar(x_p + w/2, bias_in_svi, width=w, color='darkorange',
              alpha=0.8, label='SVI')
axes[1,0].axhline(y=0, color='k', lw=1)
axes[1,0].set_xticks(x_p)
axes[1,0].set_xticklabels([f'{t:.2f}' for t in mats_in],
                           rotation=45, fontsize=7)
axes[1,0].set(xlabel='Maturit T (ans)', ylabel='Biais moyen (%)',
              title='Biais in-sample\n(bleu=Poly, orange=SVI)')
axes[1,0].legend(fontsize=9)
axes[1,0].grid(alpha=0.3, axis='y')

# Out-of-sample
all_out_poly = df_out_poly[df_out_poly['P_obs'] > 200]
all_out_svi = df_out_svi[df_out_svi['P_obs'] > 200]
mats_out = sorted(all_out_poly['T'].round(3).unique())
bias_out_poly, bias_out_svi = [], []
for T_sel in mats_out:
    s_p = all_out_poly[np.abs(all_out_poly['T']-T_sel)<0.015]
    s_s = all_out_svi[np.abs(all_out_svi['T']-T_sel)<0.015]

```

```

        bias_out_poly.append(np.mean(s_p['err_pct']) if len(s_p) else 0)
        bias_out_svi.append(np.mean(s_s['err_pct']) if len(s_s) else 0)

x_o = np.arange(len(mats_out))
axes[1,1].bar(x_o - w/2, bias_out_poly, width=w, color='steelblue',
              alpha=0.8, label='Polynomiale')
axes[1,1].bar(x_o + w/2, bias_out_svi, width=w, color='darkorange',
              alpha=0.8, label='SVI')
axes[1,1].axhline(y=0, color='k', lw=1)
axes[1,1].set_xticks(x_o)
axes[1,1].set_xticklabels([f'{t:.2f}' for t in mats_out],
                           rotation=45, fontsize=7)
axes[1,1].set_xlabel('Maturit T (ans)', ylabel='Biais moyen (%)',
                     title='Biais out-of-sample\n(bleu=Poly, orange=SVI)')
axes[1,1].legend(fontsize=9)
axes[1,1].grid(alpha=0.3, axis='y')

# MAE globale rsume (barres horizontales)
models = ['Poly\nin-sample', 'SVI\nin-sample',
          'Poly\nout-sample', 'SVI\nout-sample']
liq_ip = df_in_poly[df_in_poly['P_obs'] > 200]
liq_is = df_in_svi[df_in_svi['P_obs'] > 200]
liq_op = df_out_poly[df_out_poly['P_obs'] > 200]
liq_os = df_out_svi[df_out_svi['P_obs'] > 200]
maes = [
    np.mean(np.abs(liq_ip['err_pct'])),
    np.mean(np.abs(liq_is['err_pct'])),
    np.mean(np.abs(liq_op['err_pct'])),
    np.mean(np.abs(liq_os['err_pct'])),
]
bar_colors = ['steelblue', 'darkorange', 'steelblue', 'darkorange']
bars = axes[1,2].barh(models, maes, color=bar_colors, alpha=0.85,
                      edgecolor='k', lw=0.4)
for bar, val in zip(bars, maes):
    axes[1,2].text(val + 0.3, bar.get_y() + bar.get_height()/2,
                   f'{val:.1f}%', va='center', fontsize=9)
axes[1,2].set_xlabel('MAE prix relatif (%)')
axes[1,2].set_title('Rsum MAE globale\n(calls > 200 pts)')
axes[1,2].grid(alpha=0.3, axis='x')

plt.tight_layout()
plt.savefig('fig/fig_pricing_comparaison.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure pricing sauvegarde : fig/fig_pricing_comparaison.png")

def main():
    # Configuration
    data_dir = 'data'
    filepath = f'{data_dir}/data_raw.csv'
    processed_data_path = f'{data_dir}/data_processed.csv'
    rate_ECB_data = f'{data_dir}/ECB Data Portal_20260614132344.csv'

    # Input reference
    date_ref = '09/06/2026'
    spot_price = 6071.4 # Spot price of the underlying asset
    r_flat = 0.01931 # Taux sans risque spot (Ester)
    EPS = 1/52

    # Simulation parameters
    PCTS = [0.75, 0.90, 1.00, 1.10, 1.20]
    mats_in = [0.277, 0.449, 0.698, 1.024, 1.522] # in-sample
    mats_out = [0.353, 0.602, 0.851, 1.273] # out-of-sample

```

```

# Data import
option_data = parse_option_data(filepath, skip_row=5, date_ref=date_ref, spot_price=
spot_price, x_col='moneyness')
print('Option data import.')
print(f"{len(option_data)} options aprs filtres de liquidit.")

# Compute implied volatilities
option_data['implied_vol'] = option_data.apply(
    lambda row: implied_vol(
        price=row['price'],
        S=spot_price, # Spot price
        K=row['strike'],
        T=row['T'],
        r=r_flat, # Risk-free rate
        opt_type=row['type']
    ),
    axis=1
)
option_data.dropna(subset=['implied_vol'], inplace=True)
option_data = option_data[(option_data['implied_vol'] > 0) & (option_data['implied_vol'] <
1.5)]
print(f"{len(option_data)} options valides aprs calcul de la volatilit implicite.")

# Save the processed data to a new CSV file
option_data.to_csv(processed_data_path, index=False)
print(f"Option data enregistre dans '{processed_data_path}'.")

# Plot the implied volatility surface
plot_iv_surface(option_data)

# Descriptive statistics of the implied volatility surface
description_statistics(option_data)

# Polynomial calibration and selection
results, best_aic, best_bic = results_polynomial_selection(option_data, n_max=4, m_max=3,
lam=1e-4)
N_OPT = best_aic['n']
M_OPT = best_aic['m']
ALPHA_OPT = best_aic['alpha_hat']

# Plot the polynomial calibration results
plot_calibration_polynomial(option_data, N_OPT, M_OPT, ALPHA_OPT)

# Exponential calibration
MU_HIST, SIGMA_HIST = get_param_hist_sx5e()
BETA0_OPT, BETA1_OPT, rmse_exp, mae_exp, iv_hat_exp = calibrer_exp(option_data, eps=EPS,
sigma_hist=SIGMA_HIST)
plot_calibration_exp(option_data, BETA0_OPT, BETA1_OPT, SIGMA_HIST, EPS)

params_model = {
    'sigma_poly': {'alpha': ALPHA_OPT, 'n': N_OPT, 'm': M_OPT},
    'sigma_svi': {'beta0': BETA0_OPT, 'beta1': BETA1_OPT}
}

# Simulation and pricing validation
alpha_cir, beta_cir, gamma_cir = get_params_cir(rate_ECB_data, skip_row=0)

df_in = selectionner_options_test(option_data, mats_in, PCTS)
df_out = selectionner_options_test(option_data, mats_out, PCTS)

df_res_in_poly, df_res_in_svi = valider_pricing(
    df_in, "VALIDATION IN-SAMPLE",
    alpha_cir, beta_cir, gamma_cir, r_flat, spot_price,

```

```
        params_model, sigma_hist=SIGMA_HIST, eps_hist=EPS
    )
    df_res_out_poly, df_res_out_svi = valider_pricing(
        df_out, "VALIDATION OUT-OF-SAMPLE",
        alpha_cir, beta_cir, gamma_cir, r_flat, spot_price,
        params_model, sigma_hist=SIGMA_HIST, eps_hist=EPS
    )

    plot_pricing_results(
        df_res_in_poly, df_res_in_svi,
        df_res_out_poly, df_res_out_svi
    )

if __name__ == "__main__":
    main()
```

Listing 3 – Code Python